

VECTA — 進捗レポート 2026-07-26

概要

仕組み

検証

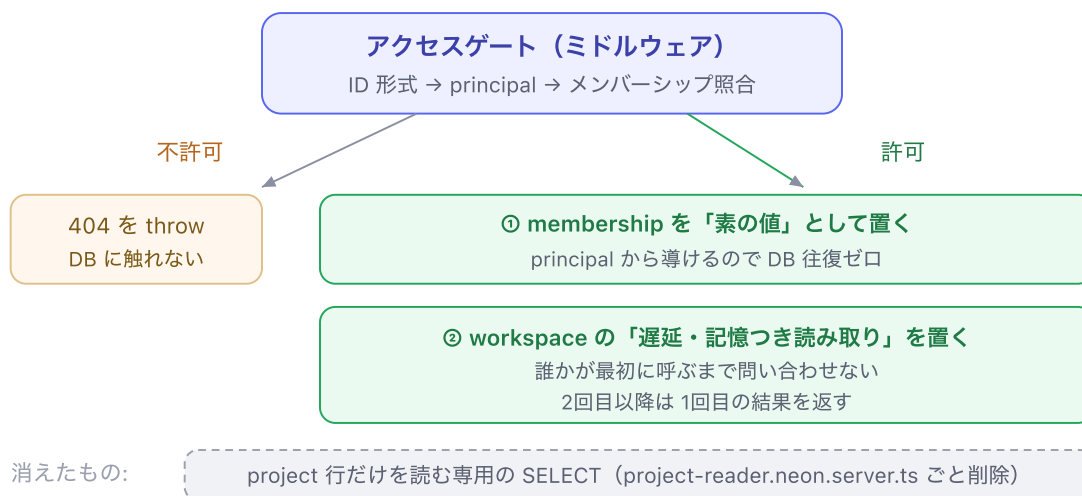
リスクと残タスク

用語集

仕組み

Abstract

- What** アクセスゲートが単独で発行していたプロジェクト行の SELECT を消し、ワークスペース一括読み取りの結果から同じ行を取り出すようにした。
- Why** 2つは同じ行を、同じ絞り込み条件で読んでいた。片方は完全に無駄だった。
- How** ゲートが context に載せる「遅延読み取り」の中身を、プロジェクト行からワークスペースに差し替えた。



拒否の判定は①②より前。だから拒否されたリクエストはプロジェクト DB に触れない

ゲートが許可後に context へ載せるもの。membership は値、workspace は遅延読み取り。

この設計の要点

- 拒否の `throw` は `next()` より前にあり、この構造は変えていない。だから「見てはいけないリクエストで子ローダーが動く」ことは起きない
- membership は問い合わせ不要なので、値として置いた。書き込みパスはこれを同期で読むだけで済む

- ワークスペース読み取りは遅延なので、読まないルート（素の `/projects/:id`）は問い合わせを1本も出さない

Detail

1. なぜ2つは同じ行だったのか

ワークスペースの一括読み取りは、8本のクエリを Neon の `db.batch` で1リクエストにまとめて送ります。その**1本目**がプロジェクトの見出し行です。

```
// packages/persistence/src/project-read-queries.ts:32
export function projectHeaderQuery(database, tenantId, projectId) {
  return database
    .select({ tenantId: tenants.id, projectId: projects.id, name: projects.name, ...
    .from(projects)
    .innerJoin(tenants, eq(tenants.id, projects.tenantId))
    .where(and(eq(projects.tenantId, tenantId), eq(projects.id, projectId)))
    .limit(1);
}
```

一方、消したゲート側の読み取りはこうでした。**絞り込み条件が完全に同じ**です。

```
// 削除した apps/web/app/server/project/project-reader.neon.server.ts:17
const [row] = await database
  .select({ id: projects.id, tenantId: projects.tenantId, name: projects.name })
  .from(projects)
  .where(and(eq(projects.tenantId, tenantId), eq(projects.id, projectId)))
  .limit(1);
```

どちらも `(tenantId, id)` の複合キーで絞っています。「テナントを跨いで ID だけで行に届くことはない」という安全性の性質も、そのまま引き継がれます。

2. なぜ3本が「逐次」だったのか

3本が並列なら回数を減らしても速くなりません。実際には次の依存があり、完全に逐次でした。

往復	何を待っていたか
① principal	Cookie セッションの検証は暗号処理だけで DB を使わない。ここが最初の問い合わせ
② project 行	①のメンバーシップ照合を通らないと発行されない

③ workspace	<code>loadProjectView</code> が <code>requireProjectAccess</code> を <code>await</code> していたので、②の完了後にしか 出せなかった
----------------	---

②を消すと③が①の直後に出せるようになります。つまり削れたのは「1本ぶんの待ち時間」まるごとです。

3. 読み出し口の書き分け

呼ぶ側が必要とするものは、実は3種類に分かれていました。分けたことで、書き込みパスからも往復が1本減りました。

関数	DB 往復	使う場所
<code>requireProjectMembership</code>	0（同期）	保存アクション、ビューのロール判定
<code>requireProjectWorkspace</code>	1（記憶つき・共有）	WBS / マスタ各画面
<code>requireProjectAccess</code>	1（上と同じものを共有）	ダッシュボード（プロジェクト名が要る画面）

```
// apps/web/app/server/project/project-access.ts:105
export async function requireProjectAccess(context) {
  const membership = requireProjectMembership(context);
  const workspace = await requireProjectWorkspace(context);
  return {
    project: { id: workspace.current.id, tenantId: membership.tenantId, name: workspace.current.name },
    membership,
  };
}
```

4. レイアウトのローダーを削った理由

`/projects/:id` のレイアウトは `{ project, membership, displayName }` を返していました。コメントには「子ルートが `useRouteLoaderData` で読むため」とありましたが、**読んでいる場所は1つもありませんでした**（全文検索で確認）。

そのまま残すと悪化します。プロジェクト一覧がリンクしているのは素の `/projects/:id`（`/wbs` へ転送するだけ）で、レイアウトのローダーはリダイレクトと**並列に走る**ため、捨てるだけのワークスペース一括読み取りが飛ぶことになります。しかもナビゲーションタブには `prefetch="intent"` が付いており、マウスを載せただけで発火します。

```
// apps/web/app/routes/project.tsx:38 - 変更後
export async function loader({ context }: Route.LoaderArgs) {
  const { principal } = await requirePrincipal(context);
  return { displayName: principal.displayName };
}
```

principal はすでに記憶済みなので、このローダーの DB 往復は**ゼロ**になりました。

5. 採らなかった選択肢

案	却下の理由
ゲートの行読み取りとワークスペース読み取りを 並列 に走らせる	回数は3本のまま。Neon の HTTP 読み取りは1リクエスト=1接続ぶんの負荷で、無駄な1本を残す理由がない。そもそも同じ行を2回読む
プロジェクト行を Cookie セッションに載せる	名前の変更が反映されなくなる。セッションは認証情報の入れ物で、業務データの複製先ではない
ダッシュボード用に 軽いクエリだけ復活 させる	今回消したものをスタブ画面のために戻すことになる。実装後のダッシュボード（EVM 表示）はどのみちワークスペースを読む
レイアウトの <code>{ project, membership }</code> を 残す	誰も読んでいないうえ、主要導線にワークスペース読み取りを引きずり込む。残す側にコストがある

参考文献

1. React Router — Middleware — reactrouter.com/how-to/middleware
2. Neon — Serverless driver / query batching — neon.com/docs/serverless/serverless-driver
3. Drizzle ORM — Neon HTTP batch — orm.drizzle.team/docs/batch-api
4. リポジトリ内: `docs/adr/0012` 系および `docs/agents/HANDOFF.md` （本変更で更新）